

K3s Networking Deep Dive

From the Day 02 request path to the CNI, node routes, CoreDNS, Service handling, and safe network change planning.

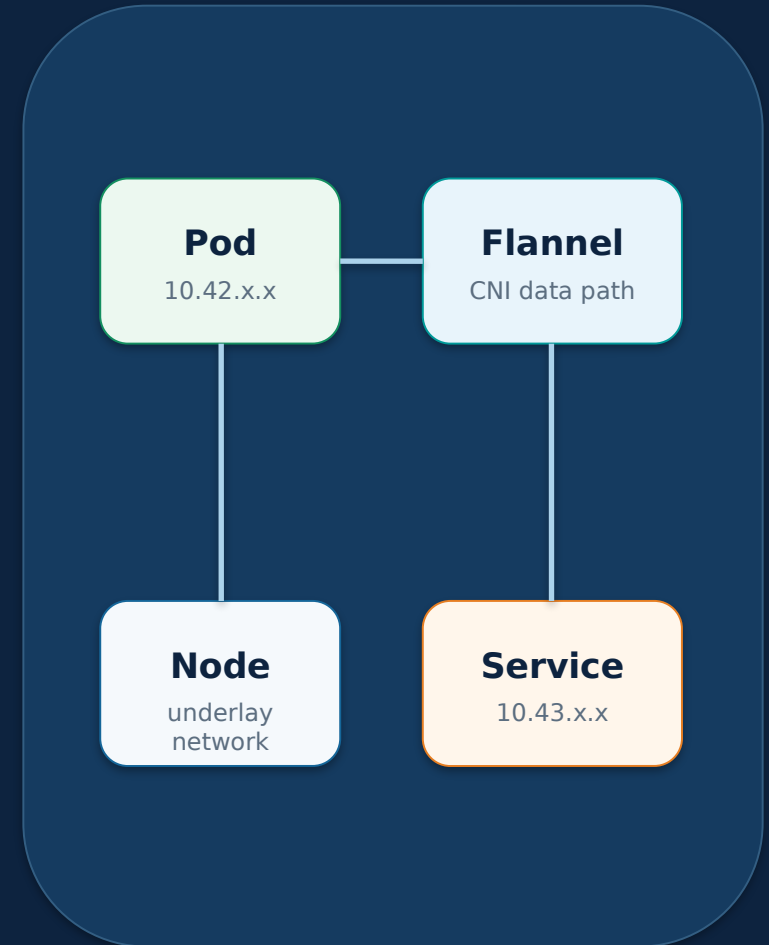
FLANNEL

COREDNS

SERVICES

POLICY

CNI



What Day 01 and Day 02 already proved

Day 01

Workloads, Services, ConfigMaps, Secrets, and RBAC identities.

Day 02

ClusterIP, NodePort, ServiceLB, ExternalName, headless DNS, and NetworkPolicy.

Today

Trace the layers beneath those successful requests and plan safe changes.

Question for the class

When dns-client called `http://web` on Day 02, which component answered the name, which component selected a Pod, and which component moved the packet?

The k3s networking stack

Application	HTTP, gRPC, database protocol
Pod network	network namespace + CNI interfaces
Node data plane	routes, overlay, forwarding
Service layer	kube-proxy rules + EndpointSlices
DNS & policy	CoreDNS + kube-router netpol

Flannel is the default CNI; it is not CoreDNS, kube-proxy, ServiceLB, or the policy controller.

Address spaces and traffic planes

Node network

Node InternalIP / ExternalIP
Underlay routes, firewall, NIC
selection

Pod network

cluster-cidr
Common k3s IPv4 default:
10.42.0.0/16

Service network

service-cidr
Common k3s IPv4 default:
10.43.0.0/16

Design rule

Pod and Service CIDRs must not overlap each other, node/LAN addresses, VPN ranges, or other routed networks.

```
kubectl get nodes -o wide  
kubectl get pod -A -o wide
```

```
kubectl get svc -A
```

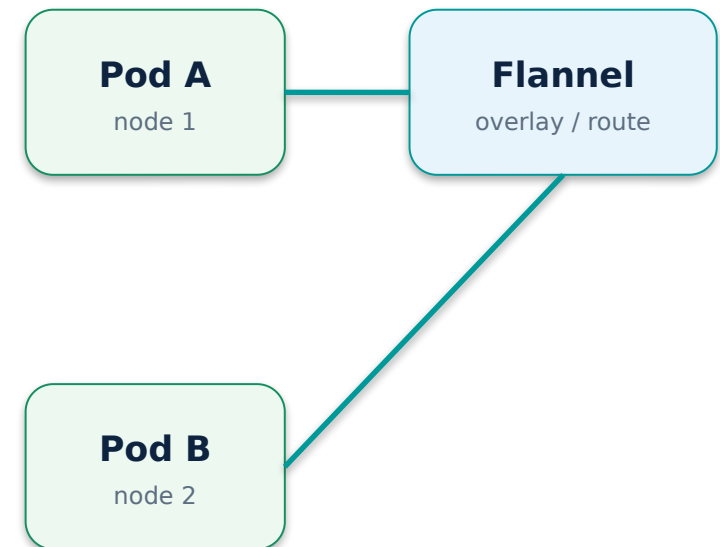
Flannel: the default k3s CNI

Lightweight Layer 3 CNI fabric; default k3s backend is VXLAN.

Connects Pods across nodes; it does not implement Service selection or DNS.

Default k3s setup enables IPv4 forwarding for Flannel.

Custom CNI stacks often need explicit forwarding configuration.



The backend determines whether the cross-node hop is an overlay or route-based path.

Flannel backend choices

Backend	Data path	Requirements / trade-offs
vxlan (default)	UDP overlay	general default; VXLAN/underlay must work
host-gw	routes via node IP	all nodes need direct Layer 2 reachability
wireguard-native	encrypted overlay	WireGuard modules required on every node
none	Flannel disabled	install one external CNI intentionally

All k3s server nodes must use identical Flannel options. Do not treat backend changes as a casual live toggle.

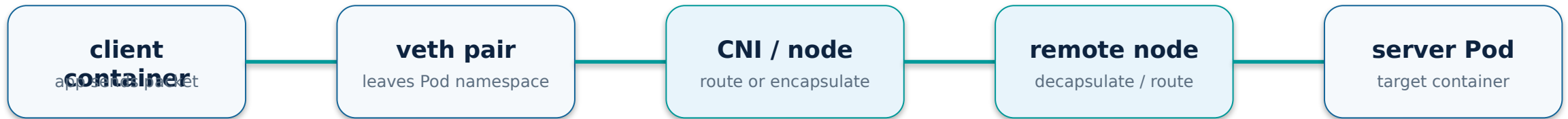
Canal, Calico, and Cilium

Stack	Connectivity / policy	Why evaluate it
Canal	Flannel connectivity + Calico policy	familiar overlay and policy combination
Calico	Calico CNI and policy	mature routing and policy choices
Cilium	eBPF CNI/data plane and policy	advanced observability and identity-aware features

Operational boundary

They are alternative network stacks. Do not run competing policy engines or layer an external CNI over default Flannel without following its installation/migration guidance.

Pod networking: follow one request



Same node

The path stays local to the node data plane.

Different node

The selected CNI backend carries traffic over the underlay.

Node networking: common hidden dependencies

Node-to-node underlay reachability and the correct node interface.

Host firewall / cloud security groups and required CNI/control-plane ports.

MTU after overlay encapsulation, reverse-path filtering, and IP forwarding.

Multi-homed node IP selection: validate --node-ip and --flannel-iface assumptions.

No overlap between LAN, VPN, Pod, and Service ranges.

```
ip -br addr
ip route
ip link show flannel.1      #
VXLAN only
    kubectl get nodes -o wide
```

**Observe the actual node.
Never diagnose a data path
from a manifest alone.**

Service networking: the Day 02 path



External entry

NodePort and k3s ServiceLB add a node-level entry path before the Service; the backend selection remains Service → EndpointSlice.

DNS and CoreDNS

CoreDNS is deployed by k3s unless all servers use --disable=coredns.

Portable FQDN: service.namespace.svc.cluster.local.

Search domains make short names work in a namespace.

ExternalName returns a CNAME; headless Services return endpoint IPs.

A valid DNS answer does not prove readiness or NetworkPolicy permission.

```
kubectl exec -n $NS dns-client -- cat /etc/resolv.conf
kubectl exec -n $NS dns-client -- nslookup web.$NS.svc.cluster.local
kubectl logs -n kube-system deploy/coredns
```

Day 02 reminder

ExternalName was a CNAME and web-headless returned individual Pod IPs. The normal web Service returned one ClusterIP.

NetworkPolicy in default k3s

Kubernetes API

You write a standard NetworkPolicy selecting target Pods and permitted traffic.

k3s controller

k3s embeds the kube-router network-policy controller library.

Node enforcement

The controller programs iptables rules on nodes to enforce selected policies.

When installing an external CNI with its own policy engine, disable the k3s network-policy controller on every server. Existing kube-router rules require planned cleanup.

Network parameters: make the contract explicit

Parameter	Why it matters
cluster-cidr	allocates Pod addresses; avoid existing routed ranges
service-cidr	virtual Service IPs; avoid all node/Pod/LAN/VPN overlaps
cluster-dns / cluster-domain	Pod resolver and Service FQDN behaviour
flannel-backend / flannel-iface	cross-node path and selected host interface
MTU / IP family	overlay payload size, IPv4/IPv6 routes and firewall rules

Validate parameter compatibility before bootstrap—not after application traffic depends on it.

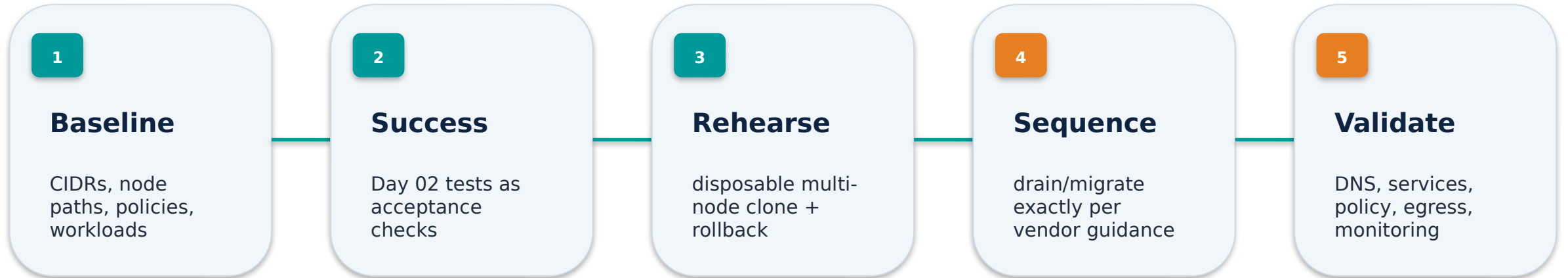
Updating network configuration: classify the risk

Risk	Examples	Required approach
Lower	app Service selector; namespace policy	test + observe + rollback manifest
Medium	host firewall; interface selection; ServiceLB port rules	maintenance plan, telemetry, rollback
High	CNI replacement; Flannel backend; Pod/Service CIDR; IP family	disposable rehearsal, backups, vendor procedure, migration plan

Hard rule

Do not casually change the training cluster's CNI, CIDRs, or Flannel backend in place. Rebuild or use the documented migration path in a disposable multi-node environment.

A safe networking change procedure



Acceptance: rerun Day 02 ClusterIP, DNS, NodePort, ServiceLB, headless, and NetworkPolicy checks

Cilium on k3s: a deliberate replacement

```
# New k3s cluster: disable the default networking pieces first  
INSTALL_K3S_EXEC='--flannel-backend=none --disable-network-policy' sh -
```

Install Cilium by its supported method and choose IPAM deliberately.

If using kube-proxy replacement, provide the Kubernetes API host and port as required.

Existing-cluster migration is specialized: use official migration guidance, a trial cluster, and tested rollback.

Do not simply install Cilium alongside default Flannel and the embedded policy controller.

Source

Cilium's k3s installation guidance explicitly starts by disabling Flannel and the built-in k3s policy controller.

Suggested labs: from observation to migration rehearsal

1 Trace Day 02 service traffic

Pods, EndpointSlices, NodePort, ServiceLB

2 Inspect Flannel

routes and overlay device on multiple nodes

3 CoreDNS failures

search paths, CNAME, empty endpoints

4 Policy proof

labels → iptables → allowed/blocked traffic

5 Backend comparison

separate VXLAN, host-gw, WireGuard clusters

6 CNI decision rehearsal

Canal/Calico/Cilium requirements + rollback

Labs 1-4 extend Day 02 safely. Labs 5-6 require a disposable multi-node cluster.

Operate the network as a system

Addressing

Document distinct node, Pod, Service, and DNS address spaces.

Discovery

Use Service names; understand ClusterIP, headless, and CNAME answers.

Control

Use policy deliberately; confirm the real enforcing data plane.

Three operating principles

Prefer stable names and Services over Pod IPs.

Treat CNI, backend, CIDR, and IP-family changes as infrastructure migrations.

Re-run the Day 02 connectivity and NetworkPolicy suite after each networking change.

Next: choose Lab 1 from LAB-ROADMAP.md